INDIVIDUAL PROJECT REPORT

Review 3 — Final Prototype, Report & Viva (9/10/2026)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

Student	Register No.	Team No.	My role in the team
Sreehari	24BDS0417	_____	Sensing layer: hardware nodes (ESP32 firmware, LoRa option), sensor simulator, satellite ground station, lost-telemetry reconstruction and model training

Team members: Laksh Ahuja (24BDS0212), Anvay Sharma (24BCE0824), Sreehari (24BDS0417).
Source code: https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK

Abstract

Cyclone monitoring starts at the sensors, and our system can only be as good as the data they deliver over lossy marine links. I built the sensing side of our three-tier edge–cloud network and the recovery of data lost on the way. This includes the sensor-node simulator with a bursty Gilbert–Elliott loss channel and a ground-truth log, the satellite ground station that synthesises four-band 16-bit frames and transmits only edge features, the ESP32 firmware for real buoy nodes with sea-level pressure correction and an alert LED/buzzer, an optional LoRa link with a serial bridge, the reconstruction module that repairs gaps with an LSTM forecast anchored to the next received reading, and the training pipeline. On packets actually lost during a live run, the reconstruction's pressure error was 0.25 hPa against 0.44 hPa for linear interpolation. For real-time concealment the forecast cut error versus holding the last value by about 27 % for wind and 53–57 % for pressure. The loss model reproduced its target loss rate to within 1 %.

Keywords: sensor networks, ESP32, LoRa, bursty packet loss, LSTM imputation, synthetic satellite imagery

1. Introduction

1.1 Problem statement

Buoys and coastal stations lose packets in bursts when storms bring rain fade and interference, so the monitoring centre sees gaps exactly in the hours when the readings change fastest.

1.2 Why it matters and who is affected

Tropical cyclones are a leading cause of weather-related deaths and losses worldwide [1], and Indian coastal states face severe storms most years [2]. Gaps in buoy data hurt forecasters, who lose the pressure trend that signals intensification, and coastal residents, whose warnings depend on it. In our live run 10 of 76 readings from one buoy were lost (13 %), three of them in a row right as pressure began to fall below 995 hPa (Section 5).

1.3 Objectives of my part

- Generate realistic telemetry and loss patterns, and record the ground truth so recovery can be measured.
- Provide a satellite ground station that produces large frames and sends only features.
- Design low-cost hardware nodes that are drop-in replacements for the simulator.
- Reconstruct lost readings better than straight-line interpolation, and train the model reproducibly.

2. Literature Review

Table 1: Literature most relevant to my modules

Author / Year	Method used	Result	Limitation for our problem
Gilbert, 1960 [3]	Two-state Markov burst-noise channel	Captures error bursts	Channel model only
Elliott, 1963 [4]	Error-rate estimates for burst channels	Practical parameter fitting	Not applied to IoT telemetry
Cao et al., 2018 [6]	BRITS: bidirectional RNN imputation	State-of-the-art imputation	Offline, needs whole series; heavy framework
Che et al., 2018 [7]	GRU-D with decay for missing values	Improves prediction with gaps	Clinical data; not real-time
Hochreiter & Schmidhuber, 1997 [8]	LSTM network	Learns temporal dynamics	Generic model
Holland, 1980 [5]	Parametric hurricane pressure profile	Realistic storm pressure field	Needs storm parameters
Augustin et al., 2016 [9]	Study of LoRa range and throughput	Long range at very low data rate	Low throughput; duty-cycle limits
Shelby et al., 2014 [10]	CoAP for constrained devices	Compact messaging for MCUs	No loss recovery at the receiver

Gap my part fills: imputation research (BRITS, GRU-D) works offline on complete series and needs large frameworks; channel models are rarely used to generate realistic test data for IoT recovery. My modules combine a bursty channel model with ground-truth logging and an online, lightweight reconstruction that works both after a gap closes and while a packet is overdue, on hardware costing about ₹800 per node.

3. Methodology

FIG. 1

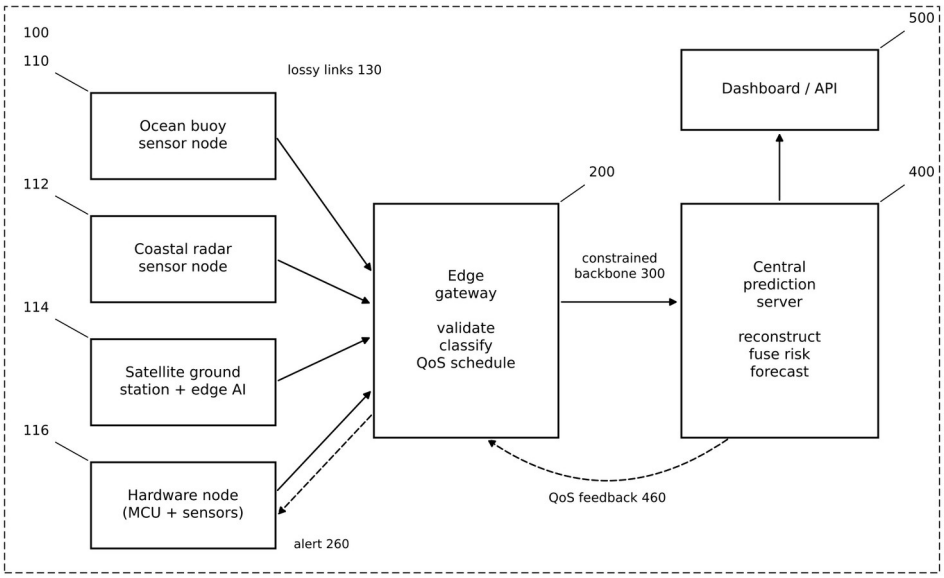


Figure 1: System architecture; my modules: sensing nodes 110–116 and the reconstruction used by server 400

3.1 Sensor node and loss model (sensor_node.py)

Each simulated node takes one reading per second from the cyclone model, numbers it, applies a loss decision, sends the surviving packet over UDP, and writes every reading — lost or not — to a truth CSV. Loss follows the Gilbert–Elliott model [3, 4]: a 'good' state with rare loss and a 'bad' (rain-fade) state with heavy loss, with transition probabilities set from the target mean loss and burstiness (Figure 2). Every fifth reading a housekeeping packet (battery, RSSI) is also sent.

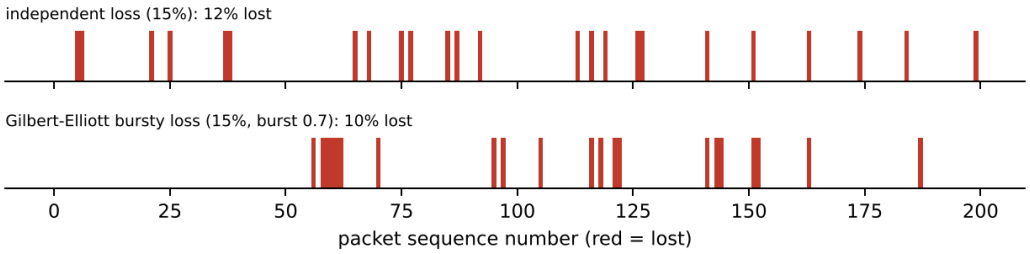


Figure 2: Independent vs bursty loss at the same target rate

3.2 Satellite ground station (satellite_station.py)

Frames are synthesised with formulas, not with an AI generator: smooth random background cloud, spiral arms from cos(2θ + 6 ln r), a central dense overcast and, above intensity 0.65, a clear eye. Four 16-bit bands (visible, infrared, water vapour, near-IR) at 512 × 512 pixels give 2.1 MB per frame (Figure 3). Each frame is passed to the feature extractor (teammate's edge-AI module) and only the ≈1 KB record is sent.

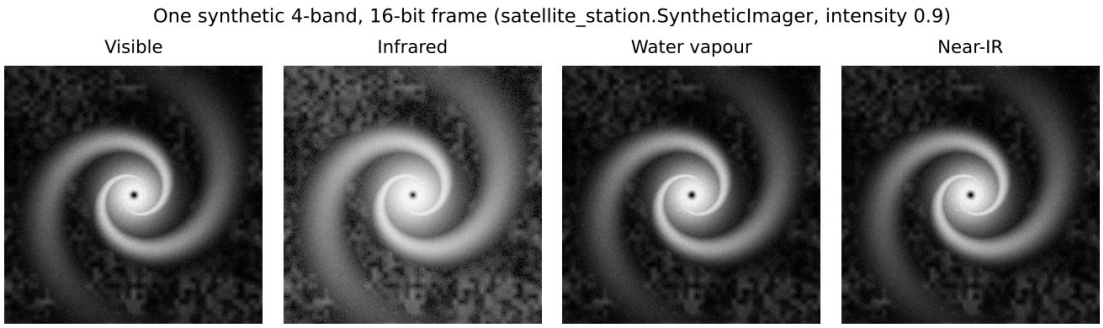


Figure 3: The four bands of one synthetic frame

3.3 Anchored LSTM reconstruction (reconstruction.py)

For a gap of n readings, the LSTM forecasts n + 1 steps from the repaired history; the error at the last step against the reading actually received is spread linearly back over the gap: estimate_k = f_k + (k / (n + 1)) · (x_next − f_(n+1)). The estimates therefore follow the learned dynamics and join the real reading without a jump (Figure 4). When no later reading exists yet, the raw forecast is used as a provisional value.

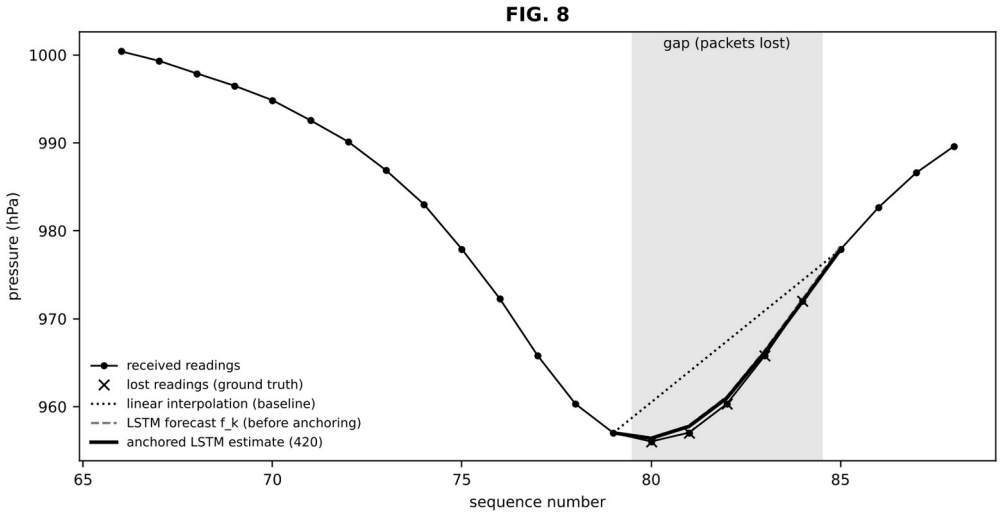


Figure 4: Anchored LSTM estimate vs linear interpolation on a gap near closest approach

3.4 Hardware node (hardware/)

An ESP32 reads a BMP280 (temperature, pressure) over I²C and a wind input (potentiometer or reed-switch cup anemometer), converts station pressure to sea-level pressure using the configured altitude, and sends the same JSON packet as the simulator. It listens on UDP 9004 and drives an LED and buzzer when the gateway reports an alert; the BOOT button toggles a storm mode for indoor demos (Figure 5).

FIG. 11

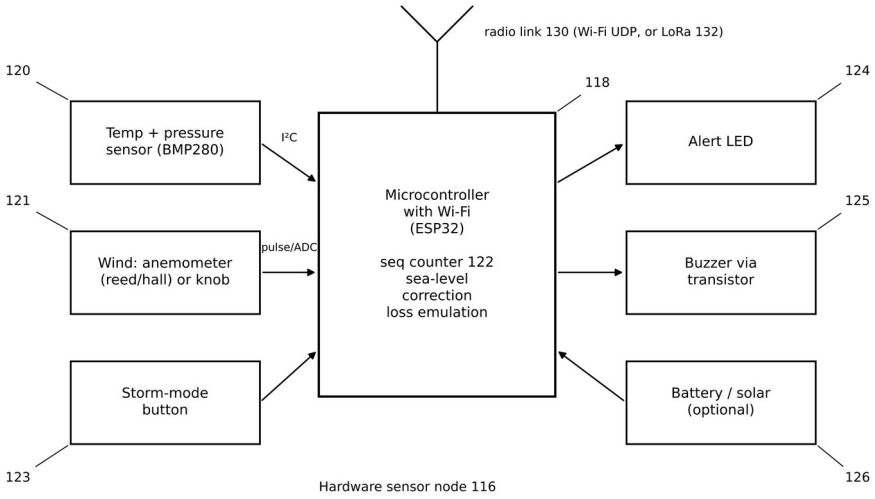


Figure 5: Hardware sensor node 116

Table 2: Wiring of the ESP32 node

Signal	Module pin	ESP32 pin	Note
Power	BMP280 VCC / GND	3V3 / GND	3.3 V only
I²C data / clock	BMP280 SDA / SCL	GPIO21 / GPIO22	address 0x76 or 0x77
Wind	Potentiometer wiper or reed switch	GPIO34 (ADC) / GPIO27 (interrupt)	WIND_POT selects
Alert LED	On-board LED	GPIO2	blinks during alert
Buzzer	BC547 base via 1 kΩ	GPIO25	buzzer on 5 V via transistor
Storm mode	BOOT button	GPIO0	ramps pressure −35 hPa, wind +70 km/h

3.5 Justification of design choices

Table 3: Design choices for my modules

Decision	Alternatives considered (approx. cost)	Chosen	Reason
Microcontroller	Arduino Uno + ESP8266 (≈₹700); Raspberry Pi Pico W (≈₹550); ESP32 (≈₹400)	ESP32	Built-in Wi-Fi, cheapest, enough RAM for JSON, deep sleep
Pressure sensor	DHT22 (no pressure, ≈₹250); BME280 (≈₹400); BMP280 (≈₹120)	BMP280	Pressure is the key cyclone signal; BME280 only adds humidity
Radio link	GSM/4G module (SIM cost); LoRa SX1278 (≈₹400); Wi-Fi (built in)	Wi-Fi, LoRa optional	Free on campus for the demo; LoRa adds a truly constrained link
Loss model	Independent coin flips; trace replay; Gilbert–Elliott	Gilbert–Elliott	Reproduces bursty rain-fade loss with two parameters
Gap repair	Linear interpolation; GRU-D / BRITS; anchored LSTM	Anchored LSTM (linear fallback)	Works online and in real time; light enough for the server and a Pi

4. Implementation

Table 4: Files I implemented

File	Lines	Key functions	What I implemented
sensor_node.py	121	run_sensor()	Readings, sequence numbers, GE/independent loss, housekeeping, truth CSV
satellite_station.py	173	SyntheticImager.frame, intensity_at, build_record, run_station	4-band frame synthesis, storm life cycle, PNG export, byte accounting
reconstruction.py	91	Reconstructor.forecast, fill_gap, make_inputs, linear_fill	Anchored gap repair, real-time forecast, linear fallback
train_models.py	109	simulate_traces, lstm_dataset, qos_dataset, main	280 random scenarios → 22,560 LSTM windows + classifier data
hardware/esp32_buoy	189	setup, loop, checkAlerts, readWindKmh	Wi-Fi node firmware
hardware/lora_*, serial_bridge.py	220	LoRa TX/RX sketches, parse_line()	Optional LoRa link → UDP
hardware/README.md	—	BOM, wiring, flashing, demo script	College-scale build guide

Table 5: Bill of materials (college-scale prices, 2025–26)

Item	Qty	Approx. ₹ each	Purpose
ESP32 DevKit V1 (ESP-WROOM-32)	2	350–450	Node microcontroller with Wi-Fi
BMP280 module (I²C)	2	100–150	Temperature + pressure
10 kΩ potentiometer / reed switch + magnet	2	10–40	Wind input (demo knob or cup anemometer)
Active buzzer + BC547 + 1 kΩ resistor	2	20–30	Alert sound
Breadboard, jumper wires, USB data cables	—	300–400	Assembly and power
Optional: SX1278 LoRa modules + antennas (pair)	1	800–1,100	Constrained radio link
Optional: 6 V solar panel + TP4056 + 18650 cell	1	450–600	Self-powered buoy
Two-node prototype (without options)		**≈1,600–2,200**	

A detail that mattered on hardware: at our campus altitude (≈216 m) a BMP280 reads about 988 hPa, which is below our 990 hPa cyclone threshold. The firmware therefore applies the barometric correction before sending:

```
float seaLevel = station / pow(1.0 - ALTITUDE_M / 44330.0, 5.255);
```

5. Testing and Results

Table 6: Benchmarks for my modules

Requirement (benchmark)	Test setup	Result	Met?
Loss model hits its target rate (±1 %)	100,000 packets at 5/10/20 %	4.98 / 9.94 / 19.73 %	Yes
Losses are bursty	Same	Mean burst 1.65–1.77 packets	Yes
Gap repair ≤ linear error for pressure	60 held-out scenarios, 3 loss rates	7–15 % lower	Yes
Gap repair ≤ linear error for wind	Same	Equal (±4 %)	Partly
Real-time forecast better than hold-last	Same	−27 % wind, −53–57 % pressure	Yes
Reconstruct real live-run losses	BUOY_07 truth CSV, 10 lost of 76	0.25 hPa vs 0.44 hPa (linear)	Yes
Frame size reduction ≥ 95 %	512² frames	2.10 MB → ≈1.04 KB (99.95 %)	Yes
Firmware compiles	g++ syntax check with stub	3/3 sketches pass; full ESP32	Partly

Requirement (benchmark)	Test setup	Result	Met?
	headers	build pending	

Table 7: Mean absolute error (km/h, hPa) under bursty loss

Loss	Variable	Linear gap repair	LSTM gap repair	Hold-last (real time)	LSTM forecast (real time)
5%	wind	1.124	1.098	2.267	1.634
5%	pressure	0.143	0.121	0.419	0.197
10%	wind	1.141	1.184	2.481	1.811
10%	pressure	0.15	0.133	0.508	0.218
20%	wind	1.121	1.131	2.255	1.656
20%	pressure	0.136	0.127	0.434	0.2

FIG. 14

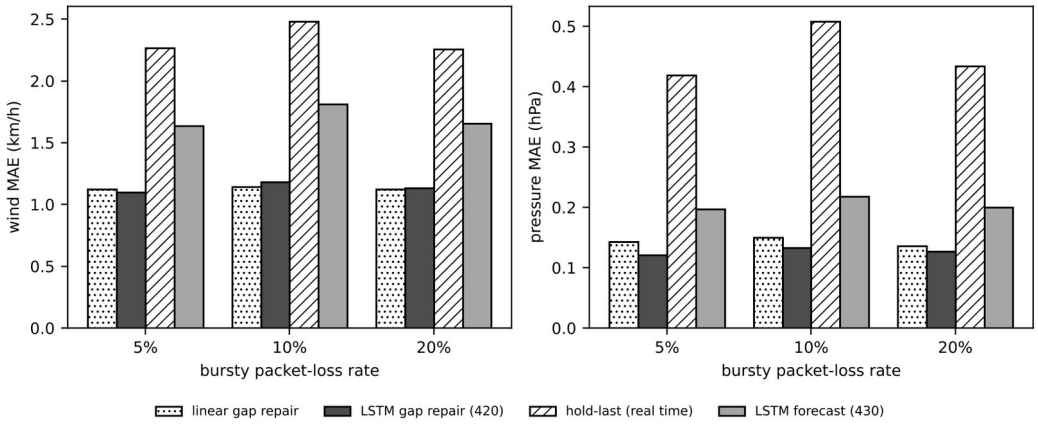


Figure 6: Reconstruction error by method and loss rate

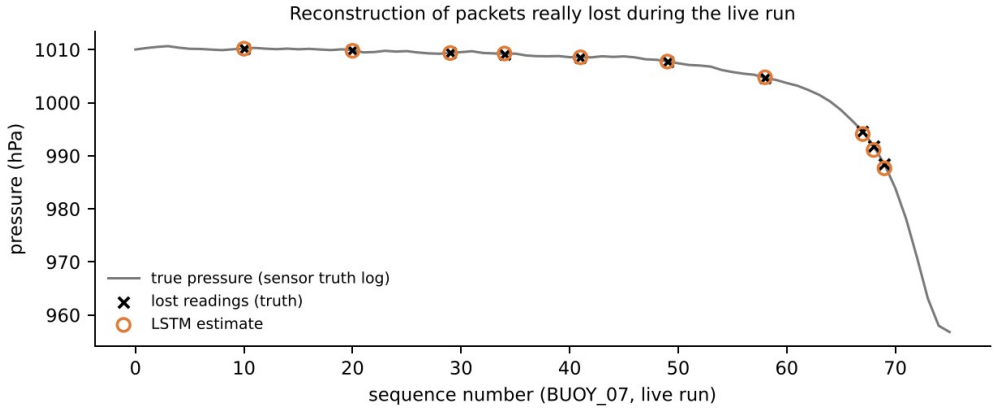


Figure 7: Packets really lost in the live run and my reconstruction

5.1 Failures found and fixed

Table 8: Failures and fixes in my modules

What failed	Diagnosis	Change	Retest result
Indoor pressure ≈988 hPa looked like a cyclone	Station pressure at 216 m altitude, not sea level	Sea-level correction with configurable ALTITUDE_M	Normal readings ≈1008 hPa
Runs not reproducible	Seeds came from Python's hash(), randomised per process	Seeds from zlib.crc32(node_id)	Identical traces every run; retraining gives identical models

What failed	Diagnosis	Change	Retest result
Satellite storms never 'cold' enough	IR mapping left cloud tops above 220 K	IR = 300 − 105·cloud^0.6; stronger overcast core	Cold cover rises with intensity; flag at ≥0.85
ESP32 toolchain could not be downloaded	Network policy blocked Arduino/Espressif servers	Syntax-checked sketches against stub headers	No syntax errors; real build planned on lab PC
LSTM gave no gain for wind	Wind gusts are close to random; a straight line is near-optimal	Kept result honest; LSTM used where it helps (pressure, real time)	Reported as limitation

6. Live Demonstration

1. `python sensor_node.py --node BUOY_07 --loss 0.15 --burst 0.6 --truth-log t.csv` — show LOST lines.
2. `python satellite_station.py --save-frames frames/` — show raw MB vs bytes sent and saved PNGs.
3. `python run_demo.py` — yellow dots on the dashboard are my reconstructions; compare with the truth CSV.
4. Hardware: ESP32 node sends live BMP280 data; turning the knob / pressing BOOT triggers CRITICAL and the alert LED + buzzer. *(Shown if the breadboard build is complete by the review.)*
5. `python train_models.py` — dataset sizes and training in ≈20 s.

6.1 Review 3 Video Plan (6 minutes)

Time	Content
0:00–2:00	Live demo: sensor LOST lines → yellow dots; satellite frames; ESP32 node if built
2:00–4:00	Test results: reconstruction MAE table, live-run truth vs estimate, fixes
4:00–5:00	Comparison with BRITS / GRU-D / Gilbert–Elliott / LoRa (Section 8)
5:00–6:00	My contribution (sensing layer, hardware, reconstruction) and field-test plan

The video starts with my name, register number, team number and project title, as required.

7. Feasibility (Five Dimensions)

Table 9: Feasibility with emphasis on my modules

Dimension	Evidence
Technical	Benchmarks in the Section 5 benchmark table met except wind gap repair (equal to baseline) and the pending full firmware build.
Economic	Per node: ESP32 ₹350–450 + BMP280 ₹100–150 + knob/reed switch, buzzer, transistor, breadboard ≈ ₹800; two-node prototype ₹1,600–2,200. Optional LoRa pair ₹800–1,100. Professional met-ocean buoys cost far more; ours is a low-cost demonstrator, not a replacement.
Social	Low cost makes dense coastal networks affordable for local bodies and colleges; feedback to be recorded: _____
Environmental	ESP32 with Wi-Fi draws on the order of 0.5 W when active; deep sleep and LoRa can cut this sharply; solar + 18650 option; boards are reusable.
Operational / Safety	3.3 V electronics only; LoRa used in India's licence-free 865–867 MHz band; antenna always attached; sealed enclosure if floated in water.

8. Comparison with Literature

Table 10: Comparison with five papers from the Review 1 survey

Parameter	Existing work	Our project	Improved?
When repair happens	Offline, full series (BRITS) [6]	Online: at gap close and in real time	Yes
Model weight	Framework RNNs (GRU-D) [7]	numpy LSTM, ≈5.1 k parameters	Yes
Imputation accuracy	Learned bidirectional models [6, 7]	Forward + anchor; no gain for wind	No (not proven)
Loss realism in testing	Burst channel theory [3, 4]	Used to generate test traffic with ground truth	Applied
Radio link	LoRa long range at low data rate [9]	Compact 30-byte CSV payload; not yet field-tested	Not yet

9. Scalability and Next Steps

- Assemble and field-test 2–3 ESP32 nodes (Wi-Fi first, then LoRa) and measure real loss and battery life.
- Train the LSTM on archived buoy data (INCOIS / NOAA NDBC) instead of simulated traces.
- Add a bidirectional model for gap repair to try to beat linear interpolation on wind.
- Weatherproof enclosure, solar charging and over-the-air firmware updates for deployment.

10. Progress since Review 2

Table 11: What changed in my modules between reviews

Area	At Review 2 (14/9)	Now (Review 3)
Sensor node	Random losses, random-walk values	Cyclone physics, bursty Gilbert–Elliott loss, housekeeping, ground-truth log
Satellite station	Not started	4-band 16-bit frame synthesis, storm life cycle, feature-only transmission
Recovery / hardware	Linear interpolation; no hardware	Anchored LSTM repair + real-time forecast; training pipeline; ESP32 + LoRa firmware, BOM

Overall completion: software 100 % of planned scope (all modules running and tested); hardware firmware complete, physical node build in progress; validation on real buoy data not yet started.

11. My Contribution

Table 12: Division of work

Tasks I did	Tasks done by teammates
Hardware: ESP32 buoy firmware, LoRa buoy/receiver sketches, serial bridge, hardware guide and BOM	Laksh Ahuja: Central prediction server; Live dashboard; QoS scheduler
Lost-telemetry reconstruction (reconstruction.py): anchored gap repair and real-time forecast	Anvay Sharma: Edge AI; Edge gateway; LSTM from scratch
Satellite ground station (satellite_station.py): 4-band frame synthesis and byte accounting	
Sensor-node simulator (sensor_node.py): readings, bursty loss, housekeeping, truth log	
Training pipeline (train_models.py)	

Evidence of my work:

- Code: the files listed in my Implementation section (https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK).
- Automated tests covering my modules, all passing (Table below; full output in

`docs/reports/evidence/pytest_output.txt`).

- Run logs from the live demonstration: logs/buoy_07.log (LOST lines), logs/satellite.log (MB → bytes), logs/truth_*.csv, train_log.txt (excerpts in Appendix B).
- Figures in this report generated from those logs by `docs/reports/make_figures.py`.

Table 13: Tests covering my modules

Automated test (tests/test_pipeline.py)	What it checks	Result
test_gilbert_elliott_mean_loss	Bursty channel's long-run loss within 8–12 % for a 10 % target	PASSED
test_reconstruction_fills_gap_and_hits_anchor	Gap estimates have the right shape and stay close to the trend	PASSED
test_esp32_json_is_accepted_by_gateway	The ESP32 firmware's exact JSON packets pass validation and classification	PASSED
test_lora_serial_bridge_parsing	LoRa CSV lines convert to valid telemetry; status and garbage lines are ignored	PASSED

> *[Insert here: screenshot of my own commits / work session and a photo of me testing — required by the rubric. Shared files (common.py, mininet_topology.py) were developed jointly.]*

12. Viva Preparation

Q: Why simulate bursty loss instead of random loss? Real marine links lose packets in clusters (rain fade); random loss would make recovery look easier than it is.

Q: Why anchor the LSTM forecast? So estimates join the reading that did arrive without a jump.

Q: Are the satellite images AI-generated? No, they are formula-based (procedural) and fully controllable.

Q: Why sea-level correction? Inland station pressure is lower; without it every reading looks cyclonic.

Q: What if the gap is longer than 30 readings? It is not reconstructed; guessing that far ahead is unreliable.

13. Conclusion

The sensing layer I built produces realistic data and realistic loss, records the truth so recovery can be measured, and runs on affordable hardware. The reconstruction keeps the pressure trend continuous through real losses and gives immediate estimates while packets are overdue. Field testing of the hardware and training on real buoy data are the next steps.

References

[1] World Meteorological Organization, *WMO Atlas of Mortality and Economic Losses from Weather, Climate and Water Extremes (1970–2019)*, WMO-No. 1267, Geneva, 2021.

[2] India Meteorological Department, "Cyclone eAtlas" and Regional Specialised Meteorological Centre (Tropical Cyclones) reports, New Delhi. [Online]. Available: https://rsmcnewdelhi.imd.gov.in

[3] E. N. Gilbert, "Capacity of a burst-noise channel," *Bell System Technical Journal*, vol. 39, no. 5, pp. 1253–1265, 1960.

[4] E. O. Elliott, "Estimates of error rates for codes on burst-noise channels," *Bell System Technical Journal*, vol. 42, no. 5, pp. 1977–1997, 1963.

[5] G. J. Holland, "An analytic model of the wind and pressure profiles in hurricanes," *Monthly*

Weather Review, vol. 108, no. 8, pp. 1212–1218, 1980.

[6] W. Cao, D. Wang, J. Li, H. Zhou, L. Li, and Y. Li, "BRITS: Bidirectional recurrent imputation for time series," in *Proc. NeurIPS*, 2018.

[7] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent neural networks for multivariate time series with missing values," *Scientific Reports*, vol. 8, art. 6085, 2018.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, "A study of LoRa: Long range & low power networks for the Internet of Things," *Sensors*, vol. 16, no. 9, art. 1466, 2016.

[10] Z. Shelby, K. Hartke, and C. Bormann, "The Constrained Application Protocol (CoAP)," IETF RFC 7252, 2014.

[11] V. F. Dvorak, "Tropical cyclone intensity analysis and forecasting from satellite imagery," *Monthly Weather Review*, vol. 103, no. 5, pp. 420–430, 1975.

Appendix A — How to Run My Modules

Command	Purpose
`python sensor_node.py --node BUOY_07 --loss 0.15 --burst 0.6 --truth-log t.csv`	Bursty-loss node with ground truth
`python satellite_station.py --save-frames frames/`	Ground station; writes IR previews
`python run_demo.py --no-sensors`	Server + gateway only, for ESP32 nodes
`python hardware/serial_bridge.py --port COM3`	LoRa receiver → gateway
`python train_models.py --quick`	Fast training run for testing

Appendix B — Evidence from the Live Run (12/10/2026)

Unedited excerpts from the log files produced by `python run_demo.py --duration 75`:

```
# logs/buoy_07.log
[BUOY_07] sensor started -> sending to 127.0.0.1:9001 (loss=12%, bursty)
[BUOY_07] seq=00011 LOST (simulated link loss)
[BUOY_07] seq=00021 LOST (simulated link loss)
[BUOY_07] seq=00030 LOST (simulated link loss)
[BUOY_07] seq=00035 LOST (simulated link loss)
[BUOY_07] seq=00042 LOST (simulated link loss)
# logs/satellite.log
[SAT_GS_01] frame 0001 intensity=0.00 sat_risk=  0.0 flag=False raw=2.10 MB -> 1073 B (99.949%
saved, 1 ms)
[SAT_GS_01] frame 0002 intensity=0.00 sat_risk=  0.0 flag=False raw=2.10 MB -> 1073 B (99.949%
saved, 1 ms)
[SAT_GS_01] frame 0014 intensity=0.85 sat_risk= 69.8 flag=True  raw=2.10 MB -> 1088 B (99.948%
saved, 12 ms)
[SAT_GS_01] frame 0015 intensity=0.93 sat_risk= 85.2 flag=True  raw=2.10 MB -> 1087 B (99.948%
saved, 13 ms)
# logs/truth_BUOY_07.csv (lost=1 rows are what the server had to reconstruct)
node,seq,wind,temp,pressure,lost
BUOY_07,11,11.24,28.44,1010.21,1
BUOY_07,21,16.55,28.6,1009.94,1
BUOY_07,30,22.48,28.68,1009.38,1
BUOY_07,35,21.88,28.7,1009.2,1
BUOY_07,42,25.36,28.73,1008.58,1
BUOY_07,50,33.66,28.49,1007.78,1
```